

Practical 8

ID-17CE049

Aim-To build a Neural network model for fashionmnist data set.

In [1]:

```
from keras.datasets import fashion_mnist
```

In [2]:

```
'''
Label Description
0   T-shirt/top
1   Trouser
2   Pullover
3   Dress
4   Coat
5   Sandal
6   Shirt
7   Sneaker
8   Bag
9   Ankle boot
'''

(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz> (<https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz>)

32768/29515 [=====] - 0s 0us/step

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz> (<https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz>)

26427392/26421880 [=====] - 0s 0us/step

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz> (<https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz>)

8192/5148 [=====] - 0s 0us/step

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz> (<https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz>)

4423680/4422102 [=====] - 0s 0us/step

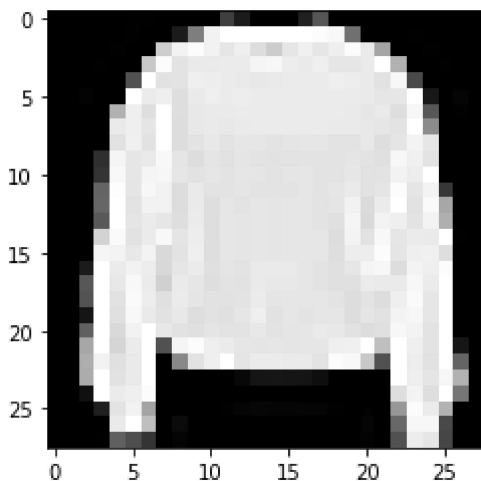
In [3]:

```
#Image training dataset is having 60,000 28 X 28 grayscale images.  
#Image testing dataset is having 10,000 28 X 28 grayscale images.  
print(x_train.shape)  
print(y_train.shape)  
print(x_test.shape)  
print(y_test.shape)
```

```
(60000, 28, 28)  
(60000,)  
(10000, 28, 28)  
(10000,)
```

In [8]:

```
import matplotlib.pyplot as plt  
plt.imshow(x_train[7,:,:].reshape(28,28),cmap='gray')  
plt.show()
```



In [6]:

```
y_train[7]
```

Out[6]:

```
2
```

In [9]:

```
#Convert labels in one hot vector  
from keras.utils import to_categorical  
y_train_encoded = to_categorical(y_train)  
y_test_encoded = to_categorical(y_test)
```

In [10]:

```
y_train_encoded[7]
```

Out[10]:

```
array([0., 0., 1., 0., 0., 0., 0., 0., 0., 0.], dtype=float32)
```

In [11]:

```
# Normalize the images.  
x_train_norm = (x_train / 255) - 0.5  
x_test_norm = (x_test / 255) - 0.5
```

In [12]:

```
print(x_train_norm.shape)  
print(x_train[1,1,1],x_train_norm[1,1,1])
```

```
(60000, 28, 28)  
0 -0.5
```

In [13]:

```
# Flatten the images.  
x_train_images = x_train_norm.reshape((-1, 784))  
x_test_images = x_test_norm.reshape((-1, 784))
```

In [14]:

```
print(x_train_images.shape)  
print(y_train_encoded.shape)  
print(x_test_images.shape)  
print(y_test_encoded.shape)
```

```
(60000, 784)  
(60000, 10)  
(10000, 784)  
(10000, 10)
```

In [26]:

```
from keras.models import Sequential  
from keras.layers import Dense  
model = Sequential()  
model.add(Dense(64, input_dim=784, activation='relu'))  
model.add(Dense(32, activation='relu'))  
model.add(Dense(10, activation='softmax'))
```

In [27]:

```
model.compile(loss='categorical_crossentropy', optimizer='SGD', metrics=['accuracy'])  
model.summary()
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
dense_10 (Dense)	(None, 64)	50240
dense_11 (Dense)	(None, 32)	2080
dense_12 (Dense)	(None, 10)	330

Total params: 52,650
Trainable params: 52,650
Non-trainable params: 0

In [28]:

```
# Training a model on Train data and at the end it will updates the weights.  
model.fit(x_train_images, y_train_encoded, epochs=30, batch_size=15)
```

```
Epoch 1/30  
4000/4000 [=====] - 4s 935us/step - loss: 0.6194  
- accuracy: 0.7830  
Epoch 2/30  
4000/4000 [=====] - 4s 928us/step - loss: 0.4425  
- accuracy: 0.8418  
Epoch 3/30  
4000/4000 [=====] - 4s 919us/step - loss: 0.4045  
- accuracy: 0.8539  
Epoch 4/30  
4000/4000 [=====] - 4s 916us/step - loss: 0.3788  
- accuracy: 0.8627  
Epoch 5/30  
4000/4000 [=====] - 4s 1ms/step - loss: 0.3626 -  
accuracy: 0.8671  
Epoch 6/30  
4000/4000 [=====] - 4s 1ms/step - loss: 0.3472 -  
accuracy: 0.8738  
Epoch 7/30  
4000/4000 [=====] - 4s 999us/step - loss: 0.3354  
- accuracy: 0.8773  
Epoch 8/30  
4000/4000 [=====] - 4s 910us/step - loss: 0.3242  
- accuracy: 0.8820  
Epoch 9/30  
4000/4000 [=====] - 4s 923us/step - loss: 0.3155  
- accuracy: 0.8851  
Epoch 10/30  
4000/4000 [=====] - 4s 923us/step - loss: 0.3067  
- accuracy: 0.8888  
Epoch 11/30  
4000/4000 [=====] - 4s 929us/step - loss: 0.2985  
- accuracy: 0.8902  
Epoch 12/30  
4000/4000 [=====] - 4s 922us/step - loss: 0.2907  
- accuracy: 0.8943  
Epoch 13/30  
4000/4000 [=====] - 4s 922us/step - loss: 0.2845  
- accuracy: 0.8961  
Epoch 14/30  
4000/4000 [=====] - 4s 932us/step - loss: 0.2787  
- accuracy: 0.8995  
Epoch 15/30  
4000/4000 [=====] - 4s 932us/step - loss: 0.2720  
- accuracy: 0.8997  
Epoch 16/30  
4000/4000 [=====] - 4s 926us/step - loss: 0.2679  
- accuracy: 0.9026  
Epoch 17/30  
4000/4000 [=====] - 4s 924us/step - loss: 0.2624  
- accuracy: 0.9040  
Epoch 18/30  
4000/4000 [=====] - 4s 930us/step - loss: 0.2570  
- accuracy: 0.9057  
Epoch 19/30
```

```

4000/4000 [=====] - 4s 927us/step - loss: 0.2510
- accuracy: 0.9091
Epoch 20/30
4000/4000 [=====] - 4s 923us/step - loss: 0.2480
- accuracy: 0.9093
Epoch 21/30
4000/4000 [=====] - 4s 920us/step - loss: 0.2431
- accuracy: 0.9106
Epoch 22/30
4000/4000 [=====] - 4s 924us/step - loss: 0.2386
- accuracy: 0.9128
Epoch 23/30
4000/4000 [=====] - 4s 921us/step - loss: 0.2350
- accuracy: 0.9130
Epoch 24/30
4000/4000 [=====] - 4s 923us/step - loss: 0.2311
- accuracy: 0.9159
Epoch 25/30
4000/4000 [=====] - 4s 925us/step - loss: 0.2275
- accuracy: 0.9158
Epoch 26/30
4000/4000 [=====] - 4s 931us/step - loss: 0.2240
- accuracy: 0.9178
Epoch 27/30
4000/4000 [=====] - 4s 923us/step - loss: 0.2217
- accuracy: 0.9183
Epoch 28/30
4000/4000 [=====] - 4s 937us/step - loss: 0.2167
- accuracy: 0.9193
Epoch 29/30
4000/4000 [=====] - 4s 920us/step - loss: 0.2150
- accuracy: 0.9216
Epoch 30/30
4000/4000 [=====] - 4s 932us/step - loss: 0.2112
- accuracy: 0.9228

```

Out[28]:

<tensorflow.python.keras.callbacks.History at 0x7f41f8ad2f28>

In [29]:

```

# Evaluate the model
scores = model.evaluate(x_test_images, y_test_encoded)
print("\nAccuracy: %.2f%%" % (scores[1]*100))

```

```

313/313 [=====] - 0s 836us/step - loss: 0.3528 - ac
curacy: 0.8784

```

Accuracy: 87.84%

In [30]:

```

model = Sequential()
model.add(Dense(120, input_dim=784, activation='relu'))
model.add(Dense(64, activation='relu'))
model.add(Dense(32, activation='relu'))
model.add(Dense(10, activation='softmax'))

```

In [31]:

```
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])  
model.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
=====		
dense_13 (Dense)	(None, 120)	94200

dense_14 (Dense)	(None, 64)	7744

dense_15 (Dense)	(None, 32)	2080

dense_16 (Dense)	(None, 10)	330
=====		
Total params: 104,354		
Trainable params: 104,354		
Non-trainable params: 0		

In [32]:

```
# Training a model on Train data and at the end it will updates the weights.  
model.fit(x_train_images, y_train_encoded, epochs=20, batch_size=15)
```

```
Epoch 1/20  
4000/4000 [=====] - 6s 1ms/step - loss: 0.4997 -  
accuracy: 0.8176  
Epoch 2/20  
4000/4000 [=====] - 7s 2ms/step - loss: 0.3824 -  
accuracy: 0.8597  
Epoch 3/20  
4000/4000 [=====] - 6s 2ms/step - loss: 0.3443 -  
accuracy: 0.8737  
Epoch 4/20  
4000/4000 [=====] - 6s 1ms/step - loss: 0.3234 -  
accuracy: 0.8806  
Epoch 5/20  
4000/4000 [=====] - 6s 1ms/step - loss: 0.3060 -  
accuracy: 0.8870  
Epoch 6/20  
4000/4000 [=====] - 6s 1ms/step - loss: 0.2894 -  
accuracy: 0.8932  
Epoch 7/20  
4000/4000 [=====] - 6s 1ms/step - loss: 0.2762 -  
accuracy: 0.8966  
Epoch 8/20  
4000/4000 [=====] - 6s 1ms/step - loss: 0.2673 -  
accuracy: 0.8998  
Epoch 9/20  
4000/4000 [=====] - 6s 1ms/step - loss: 0.2570 -  
accuracy: 0.9036  
Epoch 10/20  
4000/4000 [=====] - 6s 1ms/step - loss: 0.2500 -  
accuracy: 0.9059  
Epoch 11/20  
4000/4000 [=====] - 6s 1ms/step - loss: 0.2418 -  
accuracy: 0.9095  
Epoch 12/20  
4000/4000 [=====] - 6s 1ms/step - loss: 0.2357 -  
accuracy: 0.9111  
Epoch 13/20  
4000/4000 [=====] - 6s 1ms/step - loss: 0.2269 -  
accuracy: 0.9140  
Epoch 14/20  
4000/4000 [=====] - 6s 1ms/step - loss: 0.2222 -  
accuracy: 0.9165  
Epoch 15/20  
4000/4000 [=====] - 6s 1ms/step - loss: 0.2131 -  
accuracy: 0.9201  
Epoch 16/20  
4000/4000 [=====] - 6s 1ms/step - loss: 0.2124 -  
accuracy: 0.9194  
Epoch 17/20  
4000/4000 [=====] - 6s 1ms/step - loss: 0.2013 -  
accuracy: 0.9251  
Epoch 18/20  
4000/4000 [=====] - 6s 1ms/step - loss: 0.1999 -  
accuracy: 0.9257  
Epoch 19/20
```

```
4000/4000 [=====] - 6s 1ms/step - loss: 0.1952 -  
accuracy: 0.9258  
Epoch 20/20  
4000/4000 [=====] - 6s 1ms/step - loss: 0.1896 -  
accuracy: 0.9282
```

Out[32]:

<tensorflow.python.keras.callbacks.History at 0x7f41f7955eb8>

In [33]:

```
# Evaluate the model  
scores = model.evaluate(x_test_images, y_test_encoded)  
print("\nAccuracy: %.2f%%" % (scores[1]*100))
```

```
313/313 [=====] - 0s 936us/step - loss: 0.3698 - ac  
curacy: 0.8857
```

Accuracy: 88.57%

In []: